# Python Programming Assignment Manual

*Part 2 — Solution Set*

Reference solutions and walkthroughs for all 20 problems: Lists, Dictionaries, Tuples & Sets.

# SECTION 8 — LISTS

## 1. Create and Print a List

### Solution Code

```
n = int(input())
lst = [int(input()) for _ in range(n)] if n > 0 else []
# Alternative single-line input: lst = list(map(int, input().split()))

print(lst)
print(f"Length: {len(lst)}")
```

**Walkthrough:** We read N, then collect N integers into a list using a list comprehension. len() on the list gives its size directly — no manual counting needed.

## 2. Update Even-Indexed Elements

### Solution Code

```
def update_even_indexed(lst):
    for i in range(len(lst)):
        if i % 2 == 0:
            lst[i] *= 2
    return lst

# Example
print(update_even_indexed([1, 2, 3, 4, 5]))  # [2, 2, 6, 4, 10]
```

**Walkthrough:** range(len(lst)) gives us the index alongside each position. We check i % 2 == 0 to identify even indices and double those elements in place.

## 3. Real-World: Shopping Cart Manager

### Solution Code

```
def manage_cart(cart, action, item):
    if action == "add":
        cart.append(item)
    elif action == "remove":
        if item in cart:
            cart.remove(item)
    return cart

# Example
cart = ["Bread", "Milk"]
print(manage_cart(cart, "add", "Eggs"))     # ['Bread', 'Milk', 'Eggs']
```

```
print(manage_cart(cart, "remove", "Milk"))  # ['Bread', 'Eggs']
print(manage_cart(cart, "remove", "Cheese"))  # ['Bread', 'Eggs'] (no error)
```

**Walkthrough:** append() handles adding. For removal, we guard with 'if item in cart' first so remove() never raises a ValueError on a missing item.

## 4. Reverse a List Using Slicing

### Solution Code

```
def reverse_list(lst):
    return lst[::-1]

# Example
print(reverse_list([1, 2, 3, 4, 5]))  # [5, 4, 3, 2, 1]
print(reverse_list([]))               # []
print(reverse_list([9]))              # [9]
```

**Walkthrough:** The slice [::-1] means: start at the default start, end at the default end, but step backward by 1 — which walks the whole list from last element to first.

## 5. Flatten a Nested List

### Solution Code

```
def flatten(nested_list):
    result = []
    for element in nested_list:
        if isinstance(element, list):
            result.extend(flatten(element))
        else:
            result.append(element)
    return result

# Example
print(flatten([1, [2, 3, [4, 5]], 6]))  # [1, 2, 3, 4, 5, 6]
print(flatten([1, [], 2]))              # [1, 2]
print(flatten([[[1]]]))                 # [1]
```

**Walkthrough:** For each element we check if it's a list. If so, we recursively flatten it and extend() the result with whatever comes back. Otherwise we append the element directly. Recursion naturally handles any depth of nesting.

# SECTION 9 — DICTIONARIES

## 1. Create a Dictionary and Access Values

### Solution Code

```
fruit_prices = {"apple": 50, "banana": 20}
lookup = "banana"

print(fruit_prices[lookup])  # 20
# Safer alternative that avoids KeyError:
# print(fruit_prices.get(lookup, "Not found"))
```

**Walkthrough:** Direct indexing fruit_prices[key] returns the associated value in constant time. Using .get() instead is the safe pattern when the key might not exist.

## 2. Update and Add Key-Value Pairs

### Solution Code

```
def update_inventory(inventory, item, quantity):
    inventory[item] = inventory.get(item, 0) + quantity
    return inventory

# Example
inv = {"pen": 5}
print(update_inventory(inv, "pen", 3))      # {'pen': 8}
print(update_inventory(inv, "pencil", 4))   # {'pen': 8, 'pencil': 4}
```

**Walkthrough:** inventory.get(item, 0) returns the current quantity if the key exists, or 0 if it's new — either way we can safely add the incoming quantity in one line.

## 3. Real-World: Word Frequency Counter

### Solution Code

```
def word_frequency(sentence):
    words = sentence.lower().split()
    freq = {}
    for word in words:
        freq[word] = freq.get(word, 0) + 1
    return freq

# Example
print(word_frequency("the cat sat on the mat the cat ran"))
# {'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1, 'ran': 1}
print(word_frequency("The the THE"))
```

```
# {'the': 3}
```

**Walkthrough:** .lower() normalizes casing before splitting, so 'The' and 'the' are treated as the same word. We then use the same get()-based counting pattern as the inventory problem.

## 4. Merge Two Dictionaries with Conflict Resolution

### Solution Code

```
def merge_dicts(d1, d2):
    merged = dict(d1)
    for key, value in d2.items():
        merged[key] = merged.get(key, 0) + value
    return merged

# Example
print(merge_dicts({"a": 10, "b": 5}, {"b": 3, "c": 7}))
# {'a': 10, 'b': 8, 'c': 7}
```

**Walkthrough:** We start with a shallow copy of d1 so the original isn't mutated. Then for every key in d2, we add its value onto whatever is already in merged (0 if the key is new).

## 5. Real-World: Nested Student Records

### Solution Code

```
def average_marks(records):
    result = {}
    for student, subjects in records.items():
        total = sum(subjects.values())
        count = len(subjects)
        result[student] = round(total / count, 2)
    return result

# Example
records = {
    "Alice": {"Math": 90, "Science": 80},
    "Bob": {"Math": 70, "Science": 60, "Art": 100},
}
print(average_marks(records))
# {'Alice': 85.0, 'Bob': 76.67}
```

**Walkthrough:** For each student, subjects.values() gives all their marks. sum() over those values divided by len(subjects) gives the average, which we round to 2 decimal places.

# SECTION 10 — TUPLES

## 1. Create a Tuple and Access Elements

### Solution Code

```
t = (10, 20, 30, 40)
index = 2
print(t[index])  # 30
```

**Walkthrough:** Tuple indexing uses the exact same square-bracket syntax as lists: t[index]. The tuple itself remains unchanged since tuples are immutable.

## 2. Tuple Packing and Unpacking

### Solution Code

```
def swap_and_pack(a, b, c):
    packed = (a, b, c)      # packing
    x, y, z = packed        # unpacking
    return (z, y, x)        # pack again, reversed

# Example
print(swap_and_pack(1, 2, 3))       # (3, 2, 1)
print(swap_and_pack("x", "y", "z")) # ('z', 'y', 'x')
```

**Walkthrough:** Packing bundles the three arguments into one tuple. Unpacking spreads a tuple's values into separate variable names in one line. We then repack them in reverse order to return.

## 3. Slicing a Tuple

### Solution Code

```
def middle_elements(t):
    return t[1:-1]

# Example
print(middle_elements((1, 2, 3, 4, 5)))  # (2, 3, 4)
print(middle_elements(()))               # ()
print(middle_elements((1, 2)))           # ()
```

**Walkthrough:** t[1:-1] starts right after index 0 and stops right before the last index, which excludes both the first and last elements. Slicing a tuple always returns a new tuple.

## 4. Real-World: GPS Coordinate Validator

### Solution Code

```
def validate_coordinates(coords):
    valid = []
    for lat, lon in coords:
        if -90 <= lat <= 90 and -180 <= lon <= 180:
            valid.append((lat, lon))
    return tuple(valid)

# Example
coords = ((45.0, 90.0), (200.0, 10.0), (-90.0, -180.0))
print(validate_coordinates(coords))
# ((45.0, 90.0), (-90.0, -180.0))
```

**Walkthrough:** We iterate the outer tuple, unpacking each inner (lat, lon) pair directly in the for-loop header. Each pair is checked against the valid ranges before being kept, and the final list is converted back into a tuple.

## 5. Count Occurrences in a Tuple of Tuples

### Solution Code

```
def tag_counts(posts):
    counts = {}
    for post in posts:
        for tag in post:
            counts[tag] = counts.get(tag, 0) + 1
    return counts

# Example
posts = (("python", "ai"), ("python", "web"), ("ai",))
print(tag_counts(posts))
# {'python': 2, 'ai': 2, 'web': 1}
print(tag_counts((("x",), ())))
# {'x': 1}
```

**Walkthrough:** The outer loop visits each post (a tuple of tags); the inner loop visits each tag within that post. We tally each tag's count using the same get()-based dictionary pattern seen earlier — an empty inner tuple simply contributes nothing.

# SECTION 11 — SETS

## 1. Create a Set and Add Elements

### Solution Code

```
values = [5, 3, 5, 1, 3]
s = set()
for v in values:
    s.add(v)

print(s)  # {1, 3, 5}
```

**Walkthrough:** We start with an empty set and call .add() for each value. Since sets only store unique elements, duplicate adds (like the repeated 5 and 3) simply have no effect.

## 2. Remove Duplicates Using a Set

### Solution Code

```
def dedupe_sorted(lst):
    return sorted(set(lst))

# Example
print(dedupe_sorted([4, 2, 4, 1, 2, 3]))  # [1, 2, 3, 4]
print(dedupe_sorted([]))                  # []
print(dedupe_sorted([9, 8, 7]))           # [7, 8, 9]
```

**Walkthrough:** set(lst) collapses all duplicates in one step. sorted() then converts that set back into a list, arranged in ascending order.

## 3. Set Operations — Union, Intersection, Difference

### Solution Code

```
def set_operations(a, b):
    union = a | b
    intersection = a & b
    difference = a - b
    return (union, intersection, difference)

# Example
print(set_operations({1, 2, 3}, {2, 3, 4}))
# ({1, 2, 3, 4}, {2, 3}, {1})
print(set_operations({1}, {2}))
# ({1, 2}, set(), {1})
```

**Walkthrough:** Python provides these directly as operators: | for union, & for intersection, and - for difference (elements in the left set but not the right). We return all three as a tuple.

## 4. Real-World: Common Friends Finder

### Solution Code

```
def mutual_friends(network, person1, person2):
    friends1 = network.get(person1, set())
    friends2 = network.get(person2, set())
    return friends1 & friends2

# Example
network = {"Alice": {"Bob", "Cara"}, "Dan": {"Bob", "Eve"}}
print(mutual_friends(network, "Alice", "Dan"))  # {'Bob'}
print(mutual_friends({"A": {"X"}, "B": {"Y"}}, "A", "B"))  # set()
```

**Walkthrough:** We safely fetch each person's friend set with .get(name, set()), which returns an empty set if the person isn't found rather than raising a KeyError. The & operator then gives exactly the friends common to both.

## 5. Real-World: Deduplicate Event Log Tags Across Sessions

### Solution Code

```
def all_unique_tags(sessions):
    result = set()
    for session_tags in sessions:
        result |= session_tags
    return result

# Example
sessions = [{"login", "click"}, {"click", "purchase"}, {"logout"}]
print(all_unique_tags(sessions))
# {'login', 'click', 'purchase', 'logout'}
print(all_unique_tags([]))       # set()
print(all_unique_tags([{"a"}, set()]))  # {'a'}
```

**Walkthrough:** Starting from an empty set, we repeatedly apply |= (union-update) with each session's tag set. Because sets discard duplicates automatically, a tag like 'click' appearing in multiple sessions only ever counts once in the final result.